



AI휴먼 — Day 8

Python 클래스와 객체 이해 및 기초 문법 통합 미니 프로그램

핵심 키워드

Class, Object, Instance,
Attribute, Method

오늘의 최종 결과물

Student 클래스를 활용한
학생 성적 관리 미니 프로그램

오늘의 학습 목표

- 1 클래스(Class)와 객체(Object)의 관계를 설명할 수 있습니다.
- 2 `__init__()` 메서드와 `self`의 역할을 이해할 수 있습니다.
- 3 속성(Attribute)과 메서드(Method)를 가진 클래스를 작성할 수 있습니다.
- 4 하나의 클래스로 여러 객체를 생성할 수 있습니다.
- 5 여러 객체를 리스트에 저장하고 반복문으로 처리할 수 있습니다.
- 6 조건문, 반복문, 함수, 파일 처리와 클래스를 하나의 프로그램에 통합할 수 있습니다.

7일차 복습: 파일·예외·모듈

지난 시간 핵심 개념

File I/O

파일 읽기와 쓰기

Exception

실행 중 발생하는 오류 상황

try-except

오류가 발생해도 프로그램이
중단되지 않도록 처리

Module

기능을 파일 단위로 나누어
재사용

import

다른 모듈의 기능을 가져오는 명령 연결

❓ 연결 질문: 데이터와 기능을 하나의 단위로 묶어서 관리할 수는 없을까?

왜 클래스가 필요한가?

학생 1명 정보를 변수로 관리한다면?

```
name1 = "김민수"  
score1 = 85  
name2 = "이서연"  
score2 = 92
```

학생 수가 늘어나면 변수도 계속 늘어납니다.

문제점

데이터 분산

데이터가 흩어집니다.

관계 불명확

이름과 점수의 관계가 코드만 보아서는
명확하지 않을 수 있습니다.

기능 분리

학생 관련 기능도 따로 관리해야 합니다.

❏ 클래스는 관련 데이터와 기능을 하나의 설계도로 묶는 방법입니다.

핵심 용어 5가지

Class (클래스)

객체를 만들기 위한 설계도

Object (객체)

프로그램 안에서 데이터와 기능을 가진 대상

Instance (인스턴스)

특정 클래스로 실제 생성된 객체

Attribute (속성)

객체가 가지는 데이터

Method (메서드)

객체가 수행하는 기능

예시

Student 클래스 → student1, student2 객체

속성 → name, score

메서드 → show(), grade()

클래스와 객체를 현실 세계로 이해하기

붕어빵 틀 = 클래스

붕어빵 = 객체

붕어빵 틀은 여러 개의 붕어빵을 만들 수 있습니다.

Student 클래스도 여러 학생 객체를 만들 수 있습니다.

[개념 그림] Student 클래스

Student 클래스

└ student1: 이름=김민수, 점수=85

└ student2: 이름=이서연, 점수=92

└ student3: 이름=박지훈, 점수=76

객체지향 프로그래밍

OOP (Object-Oriented Programming)

핵심 컨셉

→ 프로그램을 여러 객체의 조합으로 바라봅니다.

→ 객체는 자신의 데이터와 기능을 함께 가질 수 있습니다.

→ 관련 코드를 하나의 단위로 묶어 관리하기 쉽습니다.

가장 기본적인 클래스 문법

```
class Student:  
    pass
```

설명

class

클래스를 정의하는 키워드

Student

클래스 이름

콜론(:)

다음 줄부터 클래스 내부 코드

pass

아직 구현할 코드가 없을 때 사용하는 빈 문장

❏ 클래스 이름은 관례적으로 첫 글자를 대문자로 작성합니다.

__init__() 메서드란?

__init__()는 객체를 만들 때 자동으로 실행되는 생성자(Constructor)입니다.

```
class Student:  
    def __init__(self, name, score):  
        self.name = name  
        self.score = score
```

역할

→ 객체가 처음 가질 값을 설정합니다.

→ 학생 이름과 점수를 객체 내부 속성으로 저장합니다.

self는 무엇인가?

self는 메모리에 현재 생성된 객체 자신을 가리킵니다.

```
self.name = name  
self.score = score
```

오른쪽 name, score

__init__()가 전달받은 값

왼쪽 self.name, self.score

객체 내부에 저장되는 속성

예시

```
student1 = Student("김민수", 85)  
student1.name → "김민수"  
student1.score → 85
```

객체 생성하기

클래스를 정의한 후 다음과 같이 객체를 생성합니다.

```
student1 = Student("김민수", 85)  
student2 = Student("이서연", 92)
```

같은 `Student` 클래스로 만들어졌지만 각 객체는 서로 다른 데이터를 가질 수 있습니다.

```
print(student1.name)  
print(student2.score)
```

출력

```
김민수  
92
```

속성 (Attribute) 이해하기

속성은 객체가 저장하는 데이터입니다.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

student = Student("김민수", 85)
print(student.name)
print(student.score)
```

❏ 속성은 점(.) 연산자를 사용해 접근합니다. 형식: 객체이름.속성이름

메서드(Method) 만들기

메서드는 클래스 내부에 정의된 함수입니다.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def show(self):
        print(self.name, self.score)

student = Student("김민수", 85)
student.show()
```

📄 메서드 호출 형식: 객체이름.메서드이름()

일반 함수와 메서드의 차이

일반 함수

```
def get_grade(score):  
    return "PASS" if score >= 80 else "RETRY"
```

메서드

```
class Student:  
    def grade(self):  
        return "PASS" if self.score >= 80 else "RETRY"
```

차이점

→ 함수는 독립적으로 호출합니다.

→ 메서드는 특정 객체를 통해 호출합니다.

→ 메서드는 **self**를 통해 객체의 속성을 사용할 수 있습니다.

조건문을 메서드에 넣기

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def grade(self):
        if self.score >= 80:
            return "PASS"
        else:
            return "RETRY"
```

❏ 객체 데이터와 판단 기능을 하나의 클래스 안에서 관리할 수 있습니다.

show()와 grade() 함께 사용하기

```
class Student:
    def show(self):
        print(f"이름: {self.name}, 점수: {self.score}, 결과: {self.grade()}")
```

핵심

→ 메서드 안에서 다른 메서드를 호출할 수 있습니다.

→ `self.grade()`는 현재 학생 객체의 점수로 합격 여부를 계산합니다.

→ 관련 기능을 한 클래스 내부에 모을 수 있습니다.

여러 객체를 리스트에 저장하기

```
students = [  
    Student("김민수", 85),  
    Student("이서연", 92),  
    Student("박지훈", 76)  
]  
  
for student in students:  
    student.show()
```

📌 핵심 연결: List + for + Object + Method — 지금까지 배운 문법이 하나의 흐름으로 연결됩니다.

합격자만 출력하기

```
for student in students:  
    if student.score >= 80:  
        student.show()
```

또는 메서드를 활용하면

```
for student in students:  
    if student.grade() == "PASS":  
        student.show()
```

❏ 두 번째 방식은 합격 기준 판단을 **Student** 클래스 내부에 모을 수 있다는 장점이 있습니다.

평균 점수 계산하기

```
total = 0

for student in students:
    total += student.score

average = total / len(students)
print(f"평균: {average:.1f}")
```

❏ 클래스를 사용해도 기존의 리스트, 반복문, 산술 연산 문법은 그대로 사용합니다.

프로그램 구조: 입력 → 처리 → 출력

입력 (Input)

- 학생 이름
- 학생 점수

처리 (Process)

- Student 객체 생성
- 리스트 저장
- `grade()`로 합격 여부 판단
- 평균 계산

출력 (Output)

- 전체 학생 목록
- 합격자 목록
- 평균 점수
- 결과 파일

📌 클래스는 처리 단계의 구조를 더 명확하게 만드는 도구로 사용할 수 있습니다.

미니 프로그램 의사코드

[의사코드] — 클래스 정의

Student 클래스를 정의한다

이름과 점수를 속성으로 저장한다

grade 메서드를 정의한다

점수가 80 이상이면 PASS 반환

아니면 RETRY 반환

show 메서드를 정의한다

이름, 점수, 결과를 출력한다

[의사코드] — 실행 흐름

학생 객체 여러 개를 생성한다

학생 객체들을 리스트에 저장한다

리스트를 반복하면서 전체 학생 정보를 출력한다

리스트를 반복하면서 PASS 학생만 출력한다

전체 점수의 평균을 계산한다

결과를 화면에 출력한다

필요하면 텍스트 파일로 저장한다

함수와 클래스를 함께 사용하기

```
def show_pass_students(students):  
    for student in students:  
        if student.grade() == "PASS":  
            student.show()
```

역할 분리

Student 클래스

학생 1명의 데이터와 기능

show_pass_students() 함수

여러 학생 객체를 처리하는 기능

📌 이렇게 나누면 코드의 역할이 더 명확해집니다.

예외 처리와 클래스 연결하기

사용자가 점수를 입력할 때 숫자가 아닌 값을 입력할 수 있습니다.

```
try:
    score = int(input("점수: "))
except ValueError:
    print("점수는 숫자로 입력해야 합니다.")
```

그 다음에 정상적인 값으로 **Student** 객체를 생성할 수 있습니다.

📌 지난 시간의 **try-except**가 클래스 기반 프로그램에서도 그대로 사용됩니다.

자주 발생하는 오류

오류 1. self 누락

잘못된 예: `def show():`

수정: `def show(self):`

오류 2. 속성 이름 오타

`self.score`로 저장했는데 `self.scores`로 접근하면 오류 발생

오류 3. 들여쓰기 오류

메서드는 `class` 내부에 같은 기준으로 들여쓰기

오류 4. 객체 생성 시 인수 개수 불일치

`Student("김민수")` → `score` 인수가 부족함

오류 5. 메서드를 괄호 없이 사용

`student.show` → 메서드 객체 자체

`student.show()` → 메서드 실행

디버깅 순서

01

오류 메시지의 마지막 줄을 확인합니다.

02

오류가 발생한 줄 번호를 찾습니다.

03

변수명과 속성명의 오타를 확인합니다.

04

`self`가 필요한 위치에 들어갔는지 확인합니다.

05

객체 생성 시 전달한 값의 개수를 확인합니다.

06

메서드 호출에 괄호가 있는지 확인합니다.

07

한 번에 전체를 고치기보다 오류 1개씩 수정합니다.

오늘의 대표 실습 흐름

01

Student 클래스 정의

02

__init__()으로 name, score 속성 저장

03

grade() 메서드 작성

04

show() 메서드 작성

05

Student 객체 여러 개 생성

06

리스트에 객체 저장

07

반복문으로 전체 출력

08

조건문으로 합격자 출력

09

평균 계산

10

결과를 텍스트 파일로 저장

오늘의 과제 구조

Basic

- Student 클래스 생성
- 객체 2개 생성
- 속성 출력

Application

- show()와 grade() 메서드 추가
- 여러 객체를 리스트로 관리

Challenge

- 합격자만 필터링
- 전체 평균 계산
- 결과 파일 저장

📄 난이도는 기능을 하나씩 추가하는 방식으로 올라갑니다.

오늘의 핵심 정리와 다음 학습 연결

- 1 클래스는 객체를 만들기 위한 설계도입니다.
- 2 객체는 클래스로부터 실제 생성된 대상입니다.
- 3 속성은 객체의 데이터입니다.
- 4 메서드는 객체가 수행하는 기능입니다.
- 5 리스트와 반복문을 이용하면 여러 객체를 한 번에 처리할 수 있습니다.

다음 학습: NumPy

오늘은 `Student` 객체 여러 개를 리스트로 관리했습니다.

다음 시간에는 숫자 데이터를 배열(Array)로 효율적으로 처리하는 NumPy를 학습합니다.